APRS-PPM

LoRa APRS packets in half the bytes

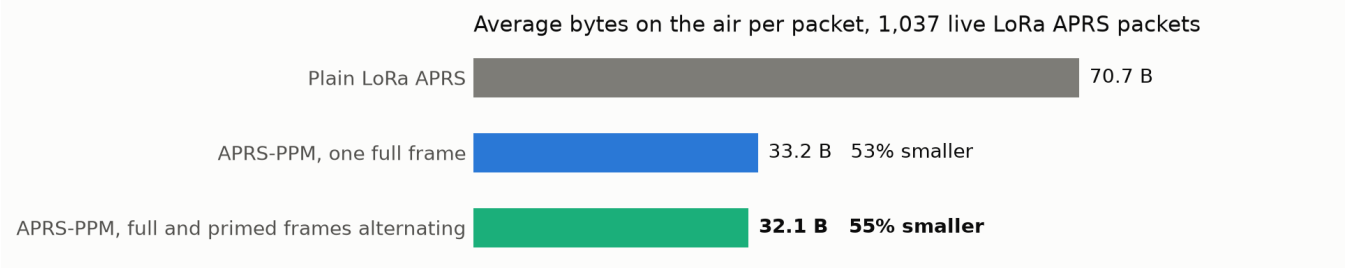
The same APRS packet, sent in about half the bytes, and turned back into the **identical text** at the other end. It extends APRS and does not replace it: stations that don't know about it carry on exactly as before.

51%
fewer bytes on the air over a full day on real radios: 80 beacons expanded of about 96 sent

55%
smaller on 1,037 live LoRa APRS packets; 47% on the ones from stations the table had never seen

0
mismatches in the frames we compared with the text sent

1 s
from a compressed beacon on the air to an ordinary packet on APRS-IS



Fresh read-only LoRa APRS-IS traffic captured on 24 October 2026; each packet is compressed and decoded back, and the bar is the average size on the air. 92% of the 1,037 packets come from stations that were in the captures the table was trained on; the 87 from stations it had never seen came out 44% smaller as one full frame and 47% with priming, in line with the 41% held-out figure. Other methods are compared in docs/alternatives-considered.md.

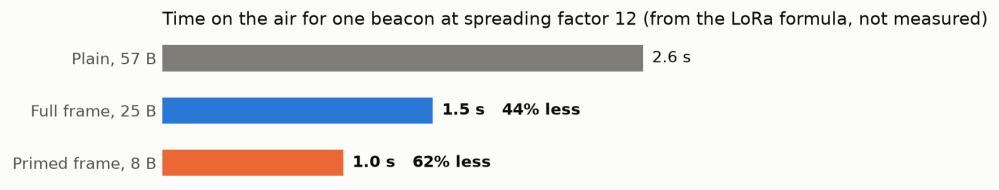
Why

A LoRa APRS packet is ordinary text, 40 to 100 bytes, sent very slowly on a channel everyone shares. At spreading factor 12 a 57-byte beacon holds the channel for about 2.6 seconds. Fewer bytes means less time on the air, fewer collisions, more stations on one channel, and longer battery life on a tracker.

How it works

APRS text is very repetitive. Nearly every packet carries the same repeater path (`WIDE1-1`), the destination field names the software (`APLRT1` for a LoRa tracker), a station repeats its own comment and symbol in every beacon, and the digits of a position barely change from one beacon to the next. Sender and receiver share a small fixed table of “what usually comes next”, so likely characters cost almost nothing and only the surprises cost bits. Nothing is learned on the air; the table ships with the software. The method is called **PPM**, *prediction by partial matching*, a text-compression technique from 1984 (Cleary and Witten) whose PPMd variant is a standard method in 7-Zip and RAR archives. Here, each character is predicted from the three before it.

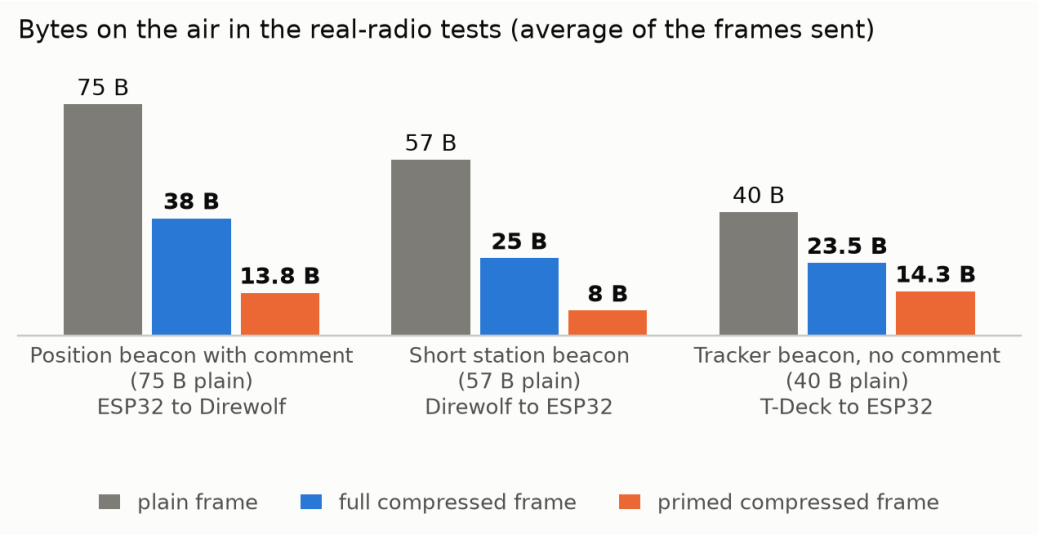
- **Full frame** (`3C FF 81`): the whole packet, compressed. Anyone with the table can read it. About **40 to 50% smaller**: 41% on stations the table had never seen, 47% over the full-day radio test.
- **Primed frame** (`3C FF C1`): coded against the same station's last full packet, so only what *changed* is sent. About **55 to 70% smaller**. A receiver that missed the reference drops it and recovers at the next full frame.
- A station sends a full frame at least every second packet by default, so a lost frame costs little. If compression would not save anything, the plain frame goes out instead.



Time saved is a little less than bytes saved, because the preamble does not shrink. In percent it is about the same at SF7.

Results on real radios

24 to 25 October 2026: an ESP32 iGate, a LILYGO T-Deck tracker and a Raspberry Pi running Direwolf with an SX1262 LoRa HAT, 433.775 MHz, SF12, 125 kHz. Every frame is listed in docs/over-the-air-test.md.



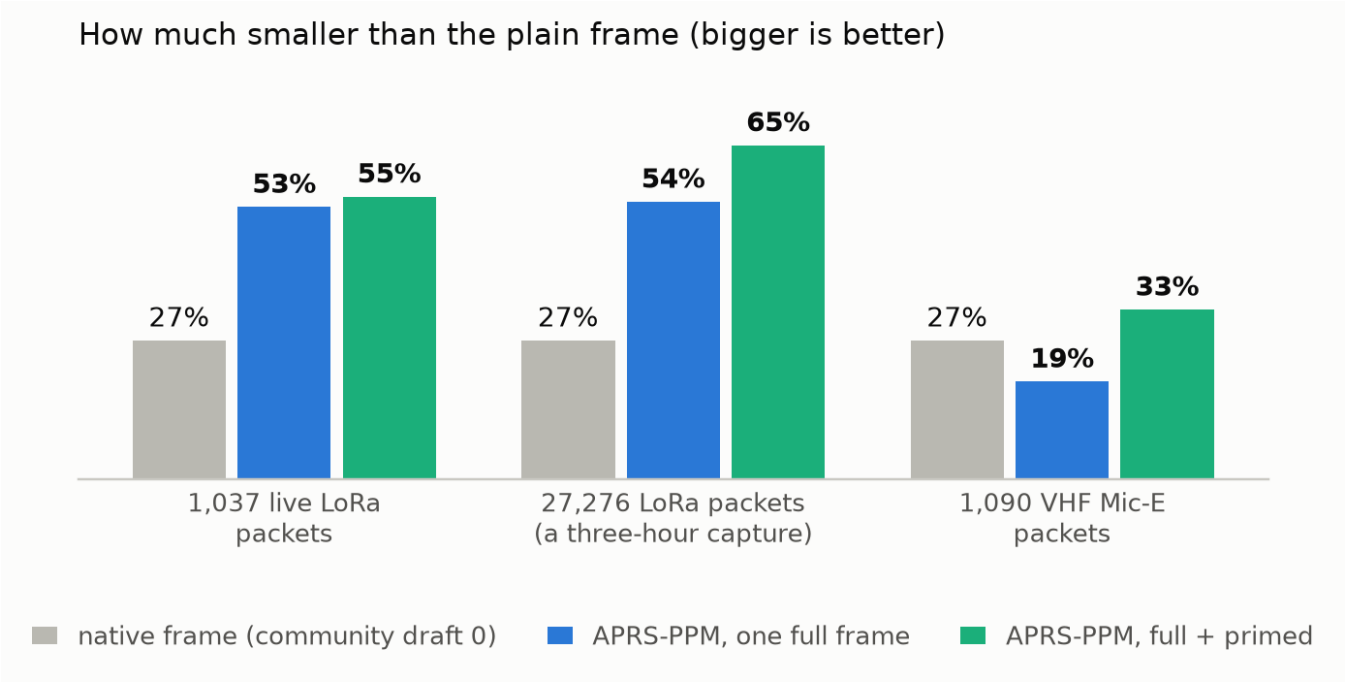
Each group is a different packet, so compare the bars within a group (plain against compressed), not between groups: a shorter packet is smaller in every format, whichever device sends it. T-Deck bars average its five beacons without the long start-up comment.

Test	Result
ESP32 sends, Direwolf receives	**12 of 12** decoded exactly. Position beacon with a comment: 75 B plain, 38 B full, 14 B primed.
Direwolf sends, ESP32 receives	**5 of 5** exactly. The gateway's short station beacon: 57 B plain, 25 B full, 8 B primed.
T-Deck sends, ESP32 receives	**6 of 6** exactly. Tracker beacon (40 B plain): full frames 42% smaller, primed frames 63 to 68% smaller.
Direwolf sends, T-Deck receives	**16 of 16** frames received; all 8 compressed ones expanded to exactly the text sent.
T-Deck to a Direwolf iGate 2 miles away, a beacon every 15 minutes	The compressed beacon was expanded and **passed to APRS-IS one second later as an ordinary packet**. Nothing downstream needed to know.
The same, for a full day (23 h 57 min)	**80 beacons expanded of about 96 sent. 51% fewer bytes on air** (1736 B against 3,515 B). About 10 were never heard; another 6 primed frames arrived but were dropped because their full frame was missed.

What exists

- **Firmware:** ESP32 iGate (feature/order3-compression), Direwolf LoRa HAT channel (feature/lora-compression) and T-Deck tracker (ppm-compression). Receive always on, transmit opt-in.

Replayed on captured packets



Native frame = a community draft (draft 0) binary layout for LoRa APRS, one packet at a time. It aims at more than size (typed fields, packed callsigns, optional receiver data), so it is a yardstick and not a rival; on Mic-E, already compact, it is a little smaller than one APRS-PPM frame. Tables: docs/alternatives-considered.md. The 27,276-packet set may overlap the model's training data. On compact VHF Mic-E packets the saving is smaller; that is reported as is.

The model was trained on 70% of the stations in a separate three-hour capture (135,299 packets) and tested on the other 30%, so the study's figures are on stations it had never seen: **41% fewer bytes** per frame without priming, about **55%** with a full frame every second packet (docs/per-country-estimate.md).

What this does not show yet

- **Needs more testing.** So far two links and three kinds of hardware: a LILYGO LoRa32 V2.1 (ESP32), a LILYGO T-Deck and a Raspberry Pi LoRa HAT running a branch of Direwolf. The longer link is about 2 miles, signal around –95 to –98 dBm at the gateway, one night, mild weather. Not yet tried: a busy channel, a really weak signal, other spreading factors, a stock iGate or tracker meeting a compressed frame.
- **Receivers without the code can't read compressed frames.** They hear and ignore them, so transmit is off by default and meant for places where every receiver that matters has the code. A compressed frame that an iGate with the code hears reaches APRS-IS as a normal packet.
- **One table for everyone.** It was trained on traffic from about 25 countries in proportion to their share of LoRa APRS, and the saving is similar across them: most countries land between about 35% and 47%, the US at 43%. The raw captures hold real callsigns and positions and are not published, only the aggregates.

The proposal repository also holds the specifications, a plain-C codec, the frozen table and 129 worked examples to check a new implementation against (spec/, reference/). These are shared so people can try the idea, not as a request. No pull request will be opened unless a maintainer says one would be welcome. MIT licence. Questions, results and problems are welcome as issues in the proposal repository.

Where the code is

What	Branch
ESP32 iGate firmware	LoRa_APRS_iGate, feature/order3-compression
Direwolf, LoRa HAT channel	direwolf, feature/lora-compression
T-Deck tracker firmware	T-Deck tracker, branch ppm-compression
Specifications, plain-C codec, test vectors, test logs	aprs-compression-proposal
Safe-testing page for the ESP32 build	EXPERIMENT.md

Questions and answers

Does an iGate with the PPM code still work on ordinary LoRa APRS packets?
Yes. A plain frame goes through exactly as before; the compressed-frame check only acts on frames that carry the compressed marker. The same iGate also reads compressed frames, so it handles both kinds on one channel.

What reaches APRS-IS when it hears a compressed frame?
The original packet, as ordinary APRS text. The iGate expands the frame back to the exact text the sender had, and from there it takes the same path as any plain packet. APRS-IS never sees a compressed frame. At our test gateway, a compressed beacon went out to APRS-IS one second after it was heard, as a normal packet with the gateway's callsign in the qA0 part of the path.

What else does an iGate do with a packet it hears: its log server, MQTT, KISS?
In the ESP32 firmware the expanded packet is the only copy there is. APRS-IS, the digipeater, KISS, MQTT, the web page list and the optional syslog server (the log server some iGates push received packets to) all take it from the same place, so they see the original text. That is read from the code. On the air we have watched the APRS-IS path, and not yet the syslog or MQTT output.

Is the text really identical?
Yes. The compression is lossless: every frame we compared with what was sent expanded to the same text, and the reference codec reproduces 129 worked examples exactly (26 full frames and 103 primed ones, each written down as bytes on the air and the text they must expand to).

Do I have to change my tracker or my iGate?
No. Receiving compressed frames is always on in the new code, and sending them is off by default. Nothing changes for a station that does not opt in.

What happens to a station that does not have the code?
It hears a compressed frame and cannot read it. We tried it with an unmodified Direwolf: it logged the frame as unparsable and carried on, without crashing or passing it to APRS-IS. That is why a compressed frame should only be sent where the receivers that matter have the code.

What if a frame is lost?
A primed frame depends on the same station's last full frame. If that was missed, the primed frame is dropped and the receiver recovers at the next full frame. That cost 6 primed frames in the 24-hour test, and it is why a full frame goes out at least every second packet.

Where does this stand?
It is an idea that is showing promise and needs more testing. It has worked on three kinds of hardware over two links, and on tens of thousands of replayed packets. More links, a busier channel, weaker signals and other spreading factors are the next things to try. The branches are there so people can try it.

How do I try it?
Use spare units, not a live station, and start with the safe-testing page. Receive is always on and transmit is opt-in, so the first step is to listen.